Baker College | GREATNESS IS EARNED HERE.

PLC Hardware

Programmable Logic Controllers

Brad Peirson
College of Information Technology and Engineering

Agenda

PLC History

PLC Classifications

PLC Sections

PLC Scan Cycle

Student Learning Outcomes

4. Explain the types and functions of generic PLC hardware
    a. Explain the functions and different variations of the five sections of a PLC: input section, output section, power supply, CPU, and programming device
    b. List the typical number of I/Os for the various size classifications of PLCs
    c. Explain the different types of PLC memory
    d. Describe the operating cycle of a PLC and how it relates to a PLC's memory

PLC History

The PLC and the Industrial Revolutions

The development of the PLC is tied to the natural development of the industrial revolutions

PLC Classifications

PLC Classifications

PLC Types:

1. Fixed
2. Modular

PLC Sizes:

1. Pico/Nano
2. Micro
3. Small
4. Large

These are *broad* classifications–and most manufacturers don't even use them

Pico/Nano PLC

- Smallest form factor
- Usually 32 I/O or less
- Good for small, single station machines

Attribution: Laserlicht, CC BY-SA 4.0, via Wikimedia Commons

Arduino Controllino Mini

Mini PLC

- 32-128 I/O
- Often expandable with additional modules
- Good for small, single station machines

Panasonic FP-XH C30T

Small PLC

- 128-960 I/O
- Large machines to entire assembly lines

Allen-Bradley CompactLogix

Large PLC

- 960+ I/O
- Large assembly lines to *entire plants*

Allen-Bradley ControlLogix

Safety PLC

- Standard PLCs are *not* rated for safety applications
- There are a number of industrial standards governing electronic safety devices–standard PLCs meet none of them
- There *are* safety-rated PLCs available from almost every manufacturer
- Pricey:
  - 5069-L306ER=$1,500
  - 5069-L306ERS2 =$2,000

Safety Relay

- If you have safety functions but don't have a safety PLC you *need* a safety relay
- Have the necessary self-monitoring, dual-channel setup
- *WAY* cheaper than safety PLCs ($100-ish)

Schneider Safety Relays

PLC Sections

PLC Sections

1. Power supply
2. Central processing unit
3. Programming device
4. Input section
5. Output section

Power Supply

- All PLCs have a power supply
- Takes input power and converts to *transistor-to-transistor (TTL)* voltage levels
  - *ALL* computers operate at TTL at the component level
  - Can be 3.3 V or 5 V–typically 5 V for PLCs
- Fixed PLC power supplies: integrated
- Modular PLC power supplies: separate module
- Multiple input voltages available: 24 VDC, 120 VAC, 240 VAC, etc.

Central Processing Unit

- Polls input section, executes the program logic, sends signals to output section
- Unlike PCs, PLC CPUs are typically specified based on memory size
  - Allen-Bradley 1769-L16ER (our trainers): 384 KB
  - Allen-Bradley 5069-L3100ERM (biggest): 10 MB
- PLCs typically do not have a lot of memory by modern computing standards, they don’t need it
- They deal with highly efficient, compiled machine code and mostly boolean logic

Programming Devices

- All PLCs require a programming device
- Some have a keypad for (limited) programming
- Older models have a dedicated programming pendant
- Modern PLCs use PC software

Input and Output Sections

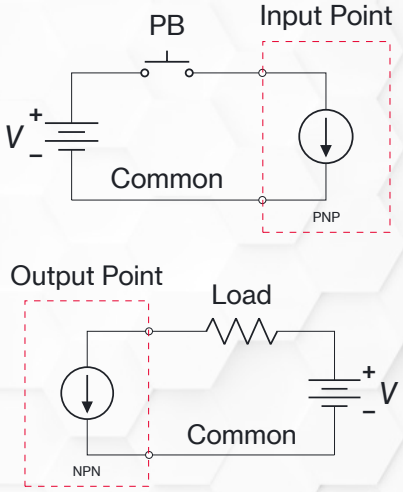
- PLCs need to be able to interface with the real world–kind of the whole point
- They do that by taking in information (inputs) to alter the state of equipment (outputs)
- The *vast* majority of PLC I/O are boolean–on or off
- That's far from the only available I/O: analog, high speed counters, communication devices, etc.

Sinking and Sourcing

- Boolean PLC I/O commonly use solid-state devices (transistors) under the hood
- It doesn't matter much for simple pushbuttons, but for modern sensors and acuators knowing the difference between sinking and sourcing is *critical*
- Sinking and sourcing refers to whether the PLC I/O point provides the external device a path to supply source (+) or a path to supply common (-)
- Solid-state devices *must match* in order to function

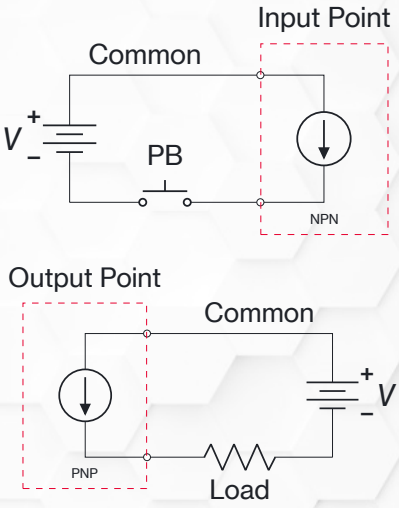
Sinking

- Sinking PLC I/O points provide a path from the I/O device, through the PLC, to the supply common (-)

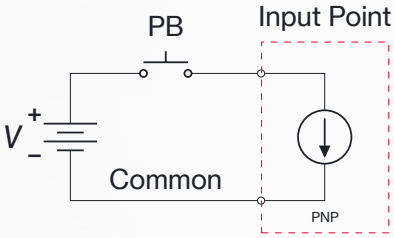


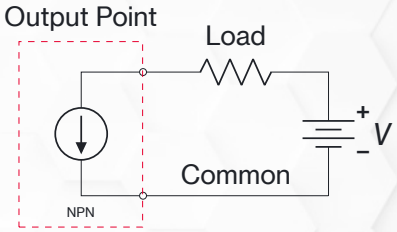
Sourcing

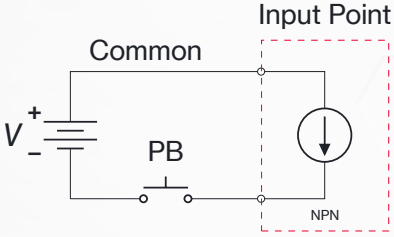
- Sourcing PLC I/O points provide a path from the I/O device, through the PLC, to the supply source (+)

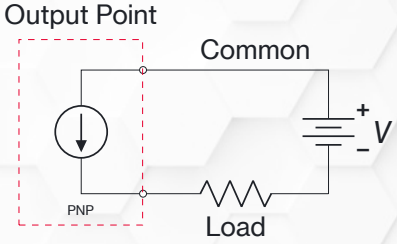


Which to Pick?









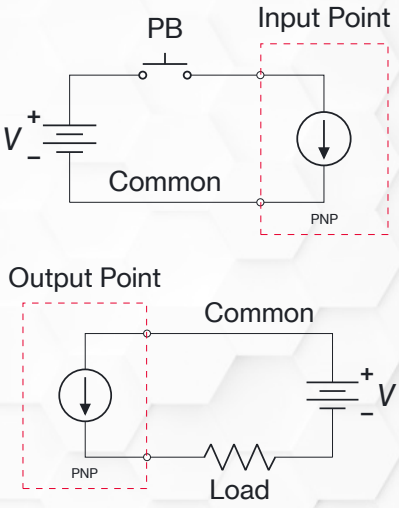
Which combination is more common in the U.S.? In Europe (IEC)? Japan?

Differing Preferences

- You *will* find a mix in most facilities–another reason it's important to understand the difference
- The U.S. prefers *sinking inputs* and *sourcing outputs*
- Older IEC standards were the opposite–they now dictate the same
- Japan still prefers sourcing inputs and sinking outputs

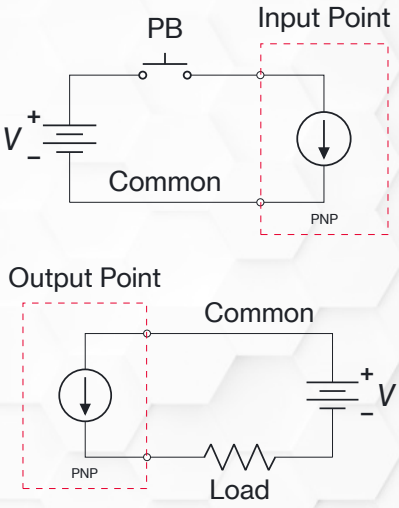
Sinking Inputs

- The machine frame is grounded–if an input wire shorts it's pulled to 0 V
- The input is forced to stay off–*fail-safe*



Sourcing Outputs

- The machine frame is grounded–if the wire to the load shorts it will pop a breaker
- The output device doesn't unintentionally actuate–*fail-safe*



Japanese Tradition

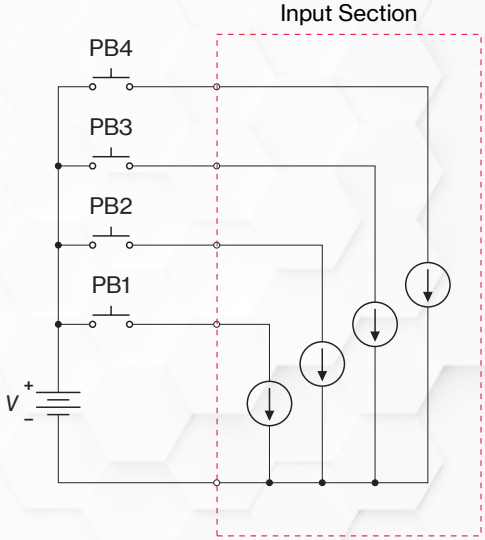
- Why use sourcing inputs/sinking outputs when the other way is fail-safe?
- Early NPN / N-channel transistors were cheaper, smaller, faster, and handled heat better than PNP / P-channel equivalents
- NPN devices sink current to 0 V–created a “normal” way of thinking
- In older standards 0 V was left floating, *not* bonded to ground–no risk of accidental trigger

Sinking and Sourcing Summary

		US Standard Practice	IEC / European (Modern)	Japanese / Asian Standard
Inputs	Module Type	Sinking Receives +24 V	Sinking Receives +24 V	Sourcing Supplies +24 V
	Required Field Device	Sourcing PNP / +24 V Switch	Sourcing PNP / +24 V Switch	Sinking NPN / 0 V Switch
	Signal Short-to-Ground	Input forced **OFF** (Fail-Safe)	Input forced **OFF** (Fail-Safe)	Input forced **ON** (False trigger risk)
Outputs	Module Type	Sourcing Outputs +24 V	Sourcing Outputs +24 V	Sinking Pulls load to 0 V
	Field Signal Line Polarity	+24 V Active	+24 V Active	0 V Common Active
	Signal Short-to-Ground	Trips Fuse / Breaker	Trips Fuse / Breaker	Load energizes unexpectedly
Primary System Rationale		Ground fault protection & positive switching	Mandated fail-safe standards (EN 60204-1)	Historical cost, speed, & heat benefits of NPN

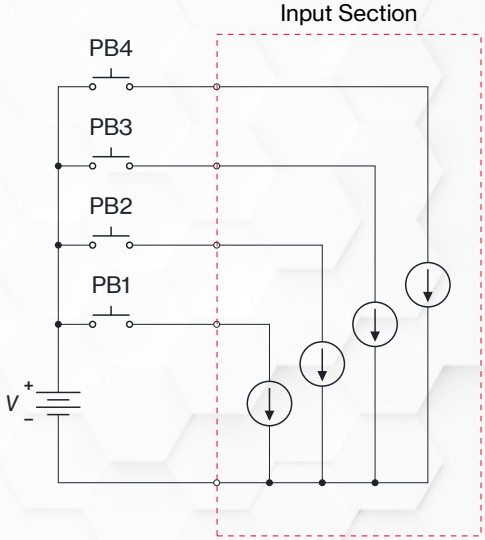
Common Terminals (1 of 6)

- I/O is a circuit–they need the return path
- Each point *could* have it's own–a lot of space/cost
- Most I/O sections use a common terminal internally wired to multiple I/O points



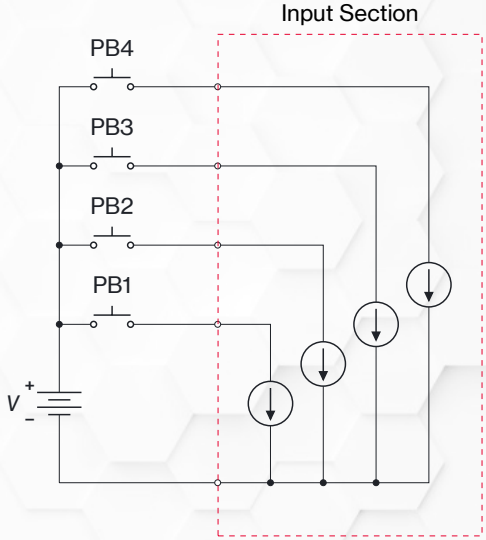
Common Terminals (2 of 6)

- Older PLCs have diagrams onboard showing what common terminal services what I/O point(s)
- There is one other concern with common terminals we need to be aware of



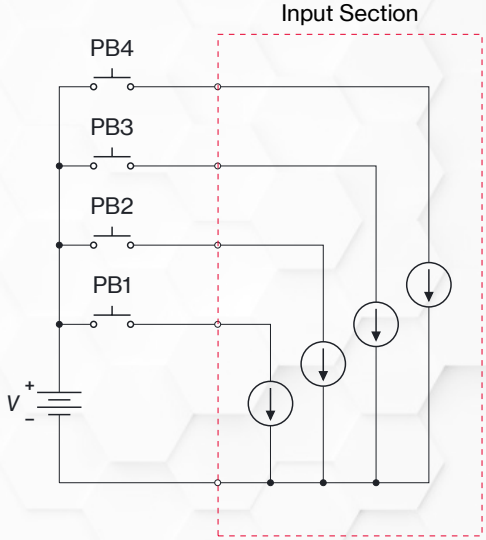
Common Terminals (3 of 6)

- What is the current through the common terminal wire?



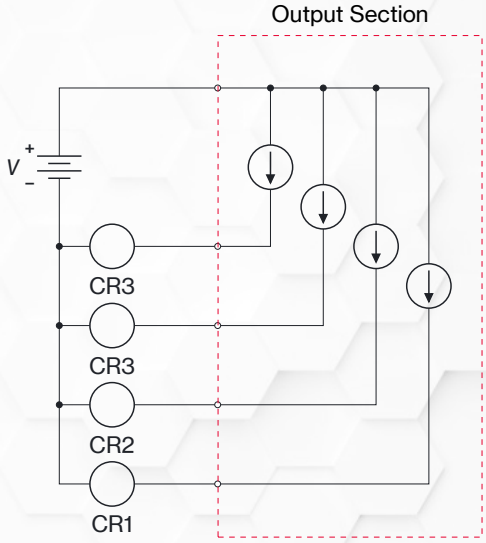
Common Terminals (4 of 6)

- It takes *all* of the current of connected I/O devices
- It needs to be sized for the entire load
- It needs to be properly protected



Common Terminals (5 of 6)

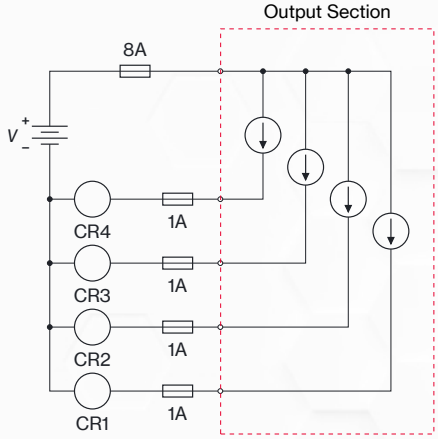
- We also need to protect against shorts
- Inputs are pulled to 0 V–not much danger
- But if we're *sourcing* outputs all of that output current is shunted to ground



Common Terminals (6 of 6)

- We *need* to protect either the (+) wire to the output common or protect each output
- Probably both–5 mm glass fuses and their holders are small and cheap

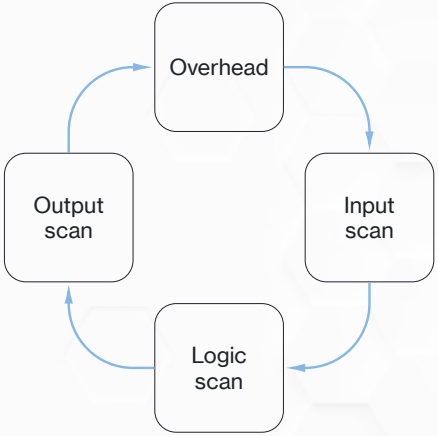
PLC Output Protection



Protecting the main source to the output common terminal *and* each individual output point is best practice

PLC Scan Cycle

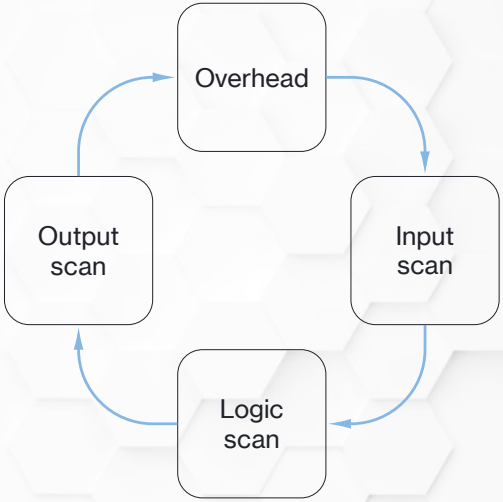
The Synchronous Scan Cycle



The *synchronous scan cycle* is a series of processing operations that the PLC performs in a continuous loop

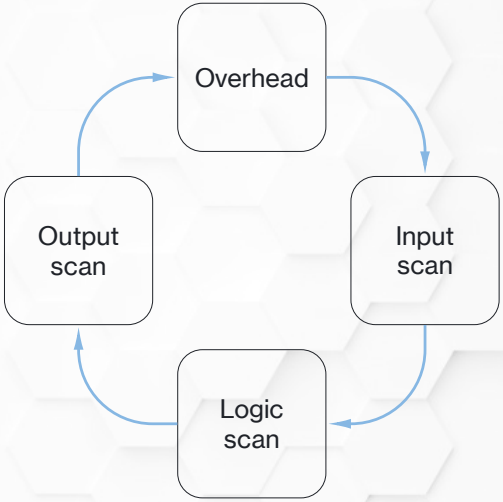
Scan Cycle - Overhead

- System diagnostics
- Communications
- Math functions



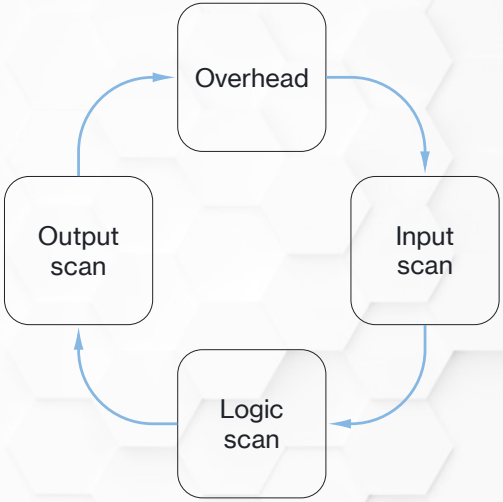
Scan Cycle - Input Scan

- Scans all input points
- Stores their value in an *input data table*



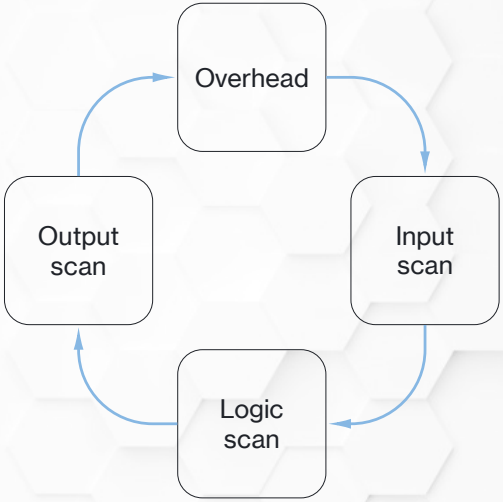
Scan Cycle - Logic Scan

- Scans the control logic left to right, top to bottom
- Reads the input data table to evaulate ladder logic
- Stores output values in *output data table*
- *NO OUTPUTS ARE TURNED ON UNTIL ALL LOGIC IS SCANNED!*

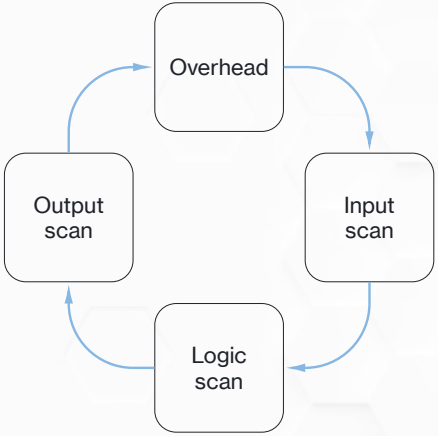


Scan Cycle - Output Scan

- Reads output values from the output data table
- Sets output values (on/off/analog) based on table value

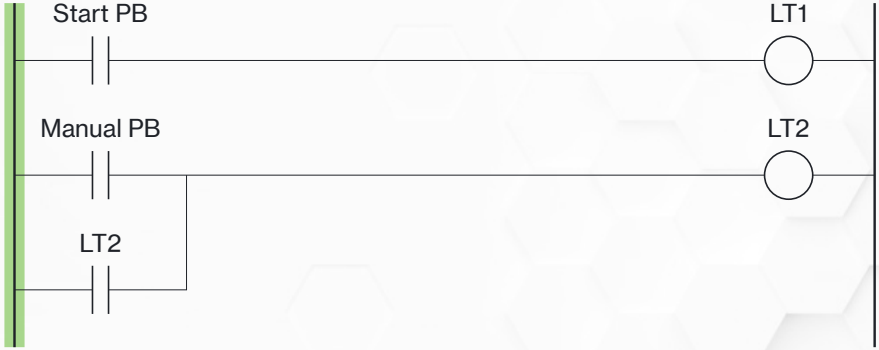


Scan Time



Even for complex programs the *scan time*, the time the PLC takes to complete one cycle, is usually under 10 ms

Program Logic Indicator

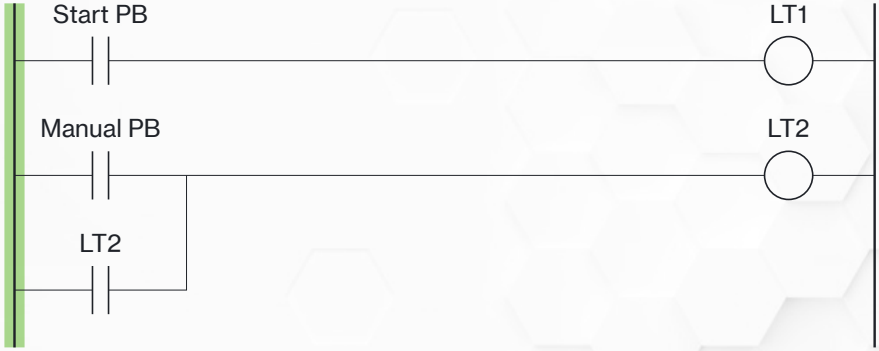


Most PLC programming softwares provide a visual indicator (green in this case) for what logic elements are currently true

Don’t Trust the Green

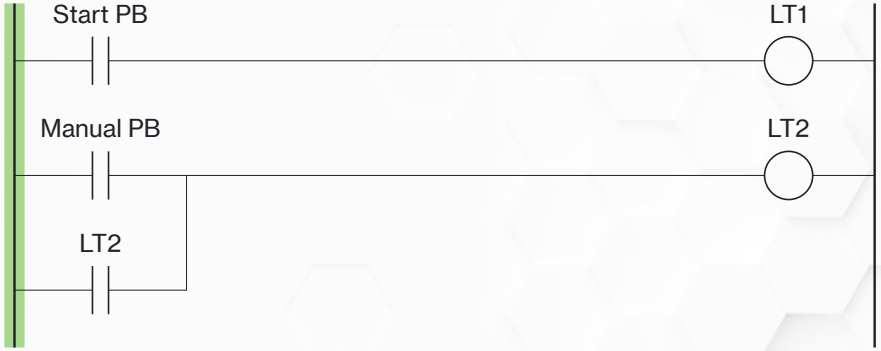
- There are a lot of reasons the green “true” indicator will be wrong
- Industrial input/output devices are *fast*
- The green indicator has to go through PLC scan time, communication time, PC processing time, and programming software time–that’s a lot of delay
- Devices can blink so fast the green never turns off
- Good for general indication, but as soon as troubleshooting starts *always* question the green

Scan Cycle Example (1 of 6)



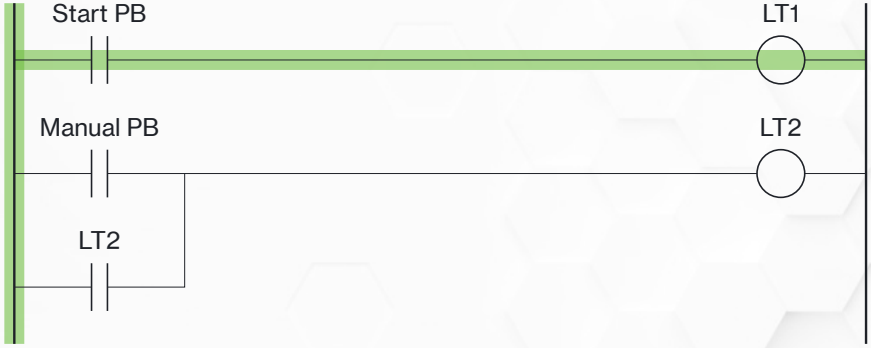
In its completely idle state *nothing* is true–the PLC *still* scans

Scan Cycle Example (2 of 6)



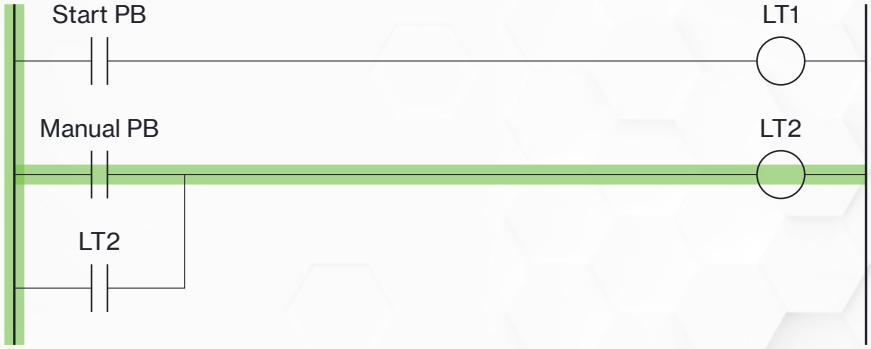
The PLC will read the input data table, see Start PB is false, and set LT1 false in the output data table. Likewise for the second rung.

Scan Cycle Example (3 of 6)



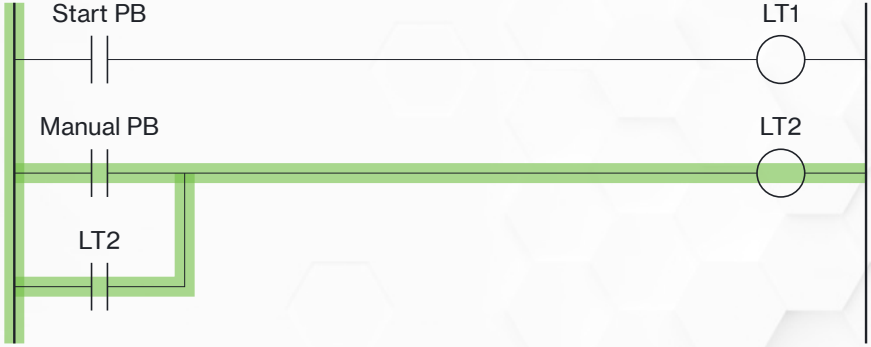
When the Start PB is pressed the PLC will read the true value from the input data table and set LT1 true in the output data table

Scan Cycle Example (4 of 6)



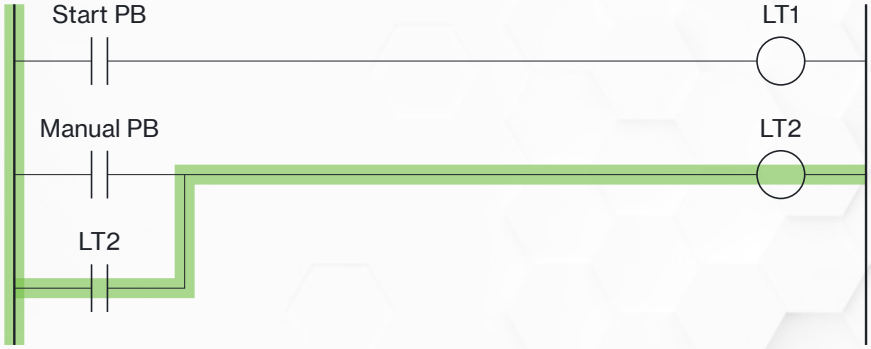
Now, let's look scan by scan–here the Start PB has been released, LT1 will be set to false. Manual PB has been pressed, LT2 will be set to true.

Scan Cycle Example (5 of 6)



When the scan comes back around, the Manual PB is still pressed. But now LT2 is *true* in the data table, so it will scan as true as well.

Scan Cycle Example (6 of 6)



When Manual PB is released, it's data table will be false. *BUT* LT2 is still true in the data table–it will be read as true and will *stay on*.

Human Inputs and Scan Time

- The entire scan cycle happens around 10 ms
- It takes a human 100-150 ms minimum to take in a stimulus (like a stack light), decide to act, and move their muscles (release the button)
- *ANY* human operated input **WILL** be on for more than one scan
- That can be a problem, we need to be aware of it, and we'll learn how to handle it when we get into programming

Copyright Notice